Table of Contents

1 Future discussions

More Kernel Optimizations for SGEMM

We have only scratched the surface of optimizing SGEMM. Other topics:

1. Occupancy
2. Contention
3. Bank Conflicts
4. Vectorized Access/Operations
5. etc. (help me fill out list, refer to SIBOEHM)

Alternative Programming Models and DSLs

In production, writing raw CUDA is rarely worth the effort.

1. Triton: tiling is at the heart of GPU optimization, so that becomes our new atomic unit instead of threads. Thread-level optimizations are optimized by a JIT compiler.
2. CUTLASS, CuTe DSL, Layout Algebra: Manually handling indexing is cumbersome and error-prone. We can compose common patterns with an algebra that can be checked at compile time.

Kernel Fusion and Specialized Kernels

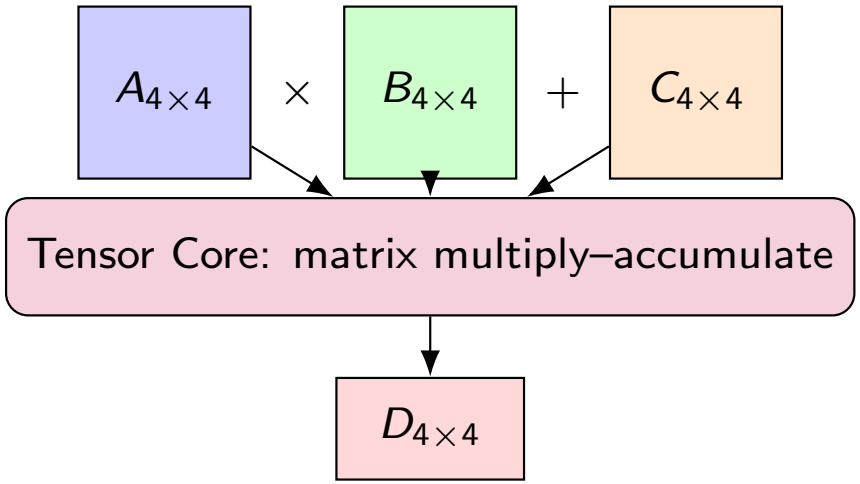
Kernels, be definition, have to read/write from DRAM. We can **fuse** kernels to increase performance.

1. Linear Layer + Nonlinearity = Fused MLP layer
2. FlashAttention series: complex fusion of the Attention Mechanism
3. Megakernels: taken to the extreme, we can fuse the operations necessary for the entire forward pass of a model into a **megakernel.**

Continuing GPU evolution

2017: Volta Tensor Cores

- Tensor Cores perform mixed-precision matrix multiply–accumulate: $D = AB + C$.
- The original Volta unit multiplies FP16 inputs and accumulates in FP16 or FP32.
- Matrix multiplication becomes an explicit hardware primitive, alongside general-purpose CUDA cores.



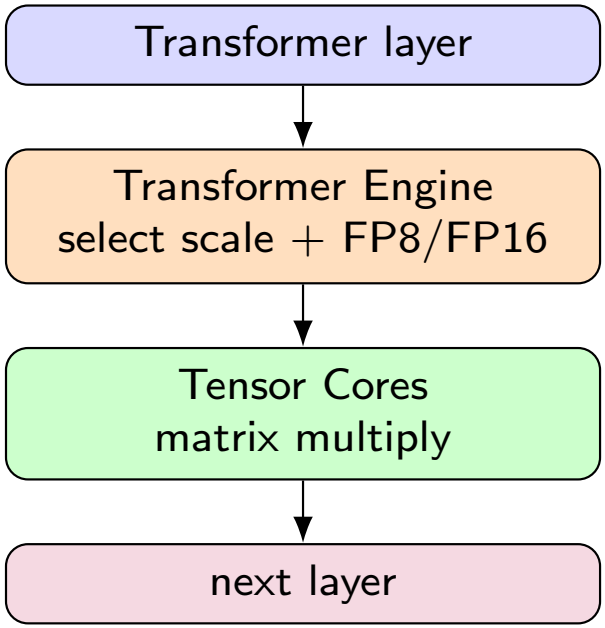
Operation shape shown for first-generation Volta Tensor Cores. Source:

NVIDIA, *Volta Architecture Whitepaper*, 2017.

Continuing GPU evolution

2020 onward: specialize around AI dataflow

- Ampere (2020) adds TF32 and structured sparsity support.
- Hopper (2022) adds FP8 Tensor Cores and a Transformer Engine that dynamically chooses FP8 or FP16 per layer.
- Blackwell (2024) adds more low-precision formats and a second-generation Transformer Engine. This is more than incremental scaling.



Source: NVIDIA, *Hopper Architecture In-Depth* (2022).

Continuing GPU evolution

2026: Vera Rubin, optimizing the rack

- The target is no longer an isolated GPU, but a complete rack-scale computing platform.
- Vera CPUs, Rubin GPUs, memory, NVLink, and networking are designed together to run one distributed AI workload.
- A high-bandwidth scale-up fabric lets many accelerators share model parameters and intermediate results.
- Performance now depends on communication, power, cooling, and software orchestration as well as matrix throughput.

Unit of optimization

Kernel

↓

GPU

↓

Server

↓

Rack-scale system

Source: NVIDIA, *Vera Rubin Architecture*, announced roadmap.

Alternative accelerator architectures

Different vendor, or different architecture?

AMD Instinct: another GPU ecosystem

AMD's datacenter GPUs use many programmable compute units, matrix instructions, and high-bandwidth memory. ROCm provides the software stack used to program them.

Strength: retains GPU flexibility while competing with CUDA.
Tradeoff: performance depends heavily on software and library support, not only the silicon.

Google TPU: systolic arrays

Data moves rhythmically through a two-dimensional grid of multiply–accumulate units. Values are reused as they pass between neighboring units instead of repeatedly returning to memory.

Strength: efficient, predictable dense tensor operations.
Tradeoff: less flexible for irregular workloads.

Sources: AMD, *Instinct Accelerators*; Jouppi et al. (2017).

Alternative accelerator architectures

Other designs change the scale or location of computation.

Cerebras: wafer-scale integration

Many compute cores, distributed SRAM, and an on-wafer fabric are placed on one large processor. More communication stays on the wafer instead of crossing between accelerator packages.

Strength: memory capacity and bandwidth close to compute.
Tradeoff: difficult manufacturing, cooling, and mapping.

NPUs: AI inside the device

NPUs sit beside CPUs and GPUs in phones, laptops, and embedded systems. They accelerate supported quantized tensor operations under strict power and thermal limits.

Strength: private, low-latency inference without the cloud.
Tradeoff: limited memory and operator support.

Sources: Cerebras, *Wafer-Scale Engine*; Sze et al. (2017).

Alternative accelerator architectures

Model ASIC: Etched Sohu

The hardware and dataflow target transformer inference rather than arbitrary kernels. Removing general-purpose machinery can devote more silicon and energy to the intended model family.

Strength: potentially much higher throughput per watt.
Tradeoff: model changes can make fixed hardware obsolete.

The design spectrum

GPU: programmable

↓

TPU/NPU: operator-focused

↓

ASIC: model-focused

More specialization ⇒ less flexibility

There is no universally best architecture

Specialization can improve performance per watt, but reduces flexibility. The right balance depends on workload stability, scale, power, and the cost of moving data.

Source: Etched (2024), "Introducing Sohu."